

EXPERIMENT NO. 11

OSSP - Operating Systems and Systems Programming

Course Code: 25CS2104E

Task 1

Multithreaded Counter Demonstrating Race Conditions

```
/*
 * Experiment 11 - Task 1: Multithreaded Shared Counter with Race Condition
 *
 * POSIX Threads Demonstrated:
 * - pthread_create(): Spawns worker threads.
 * - pthread_join(): Waits for worker threads to terminate.
 *
 * Concept:
 * - A shared global variable 'counter' is incremented concurrently by multiple threads
 *   .
 * - 'counter++' is NOT an atomic operation; it consists of 3 assembly steps:
 *   1. LOAD counter from memory into CPU register
 *   2. INCREMENT register value
 *   3. STORE register value back into memory
 * - Without synchronization, concurrent threads interleave these steps, leading to
 *   lost updates (Race Condition).
 */

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_THREADS 4
#define INCREMENTS_PER_THREAD 250000

// Shared counter without any mutex protection
long long shared_counter = 0;

void *worker_thread(void *arg) {
    long thread_id = (long)arg;
    for (int i = 0; i < INCREMENTS_PER_THREAD; i++) {
        // Critical section accessed without synchronization
        shared_counter++;
    }
    printf(" [Thread %ld] Completed %d increments.\n", thread_id, INCREMENTS_PER_THREAD);
    pthread_exit(NULL);
}

int main() {
    pthread_t threads[NUM_THREADS];
    long long expected_counter = (long long)NUM_THREADS * INCREMENTS_PER_THREAD;

    printf("=== Multithreaded Counter: Race Condition Demonstration ===\n");
    printf("Number of threads:      %d\n", NUM_THREADS);
    printf("Increments per thread:    %d\n", INCREMENTS_PER_THREAD);
    printf("Expected Final Counter:  %lld\n\n", expected_counter);
    printf("Starting worker threads...\n");

    // Spawn threads using pthread_create
    for (long t = 0; t < NUM_THREADS; t++) {
        int rc = pthread_create(&threads[t], NULL, worker_thread, (void *)t);
        if (rc != 0) {
            perror("pthread_create failed");
            exit(EXIT_FAILURE);
        }
    }

    // Wait for all threads using pthread_join
    for (int t = 0; t < NUM_THREADS; t++) {
        pthread_join(threads[t], NULL);
    }

    long long lost_updates = expected_counter - shared_counter;

    printf("\n=== Execution Results ===\n");
    printf("Expected Counter:  %lld\n", expected_counter);
```

```

printf("Actual Counter:    %lld\n", shared_counter);
printf("Lost Updates:      %lld\n", lost_updates);

if (shared_counter != expected_counter) {
    printf("RESULT: RACE CONDITION DEMONSTRATED! Final value is inconsistent.\n");
} else {
    printf("RESULT: All updates succeeded (rare timing alignment).\n");
}

return 0;
}

```

Output:



```

zsh - race_condition

rishivarma@MacBook-Air ~ % ./race_condition
=== Multithreaded Counter: Race Condition Demonstration ===
Number of threads:      4
Increments per thread:  250000
Expected Final Counter: 1000000
Starting worker threads...
[Thread 1] Completed 250000 increments.
[Thread 0] Completed 250000 increments.
[Thread 2] Completed 250000 increments.
[Thread 3] Completed 250000 increments.
=== Execution Results ===
Expected Counter: 1000000
Actual Counter: 300528
Lost Updates: 699472
RESULT: RACE CONDITION DEMONSTRATED! Final value is inconsistent.
rishivarma@MacBook-Air ~ %

```

Task 2

Mutex-Protected Counter and Synchronization Comparison

```

/*
 * Experiment 11 - Task 2: Multithreaded Counter Synchronized with Mutex
 *
 * POSIX Mutex Functions Demonstrated:
 * - pthread_mutex_init(): Initializes the mutual exclusion lock.
 * - pthread_mutex_lock(): Acquires lock before entering critical section (blocks if held).
 * - pthread_mutex_unlock(): Releases lock upon exiting critical section.
 * - pthread_mutex_destroy(): Cleans up and destroys mutex object.
 *
 * Concept:
 * - Enforces Mutual Exclusion: Only ONE thread at a time can execute 'shared_counter++'.
 * - Completely eliminates race conditions and guarantees consistent results.
 */

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_THREADS 4
#define INCREMENTS_PER_THREAD 250000

// Shared counter protected by mutex
long long shared_counter = 0;
pthread_mutex_t counter_mutex;

void *worker_thread(void *arg) {
    long thread_id = (long)arg;
    for (int i = 0; i < INCREMENTS_PER_THREAD; i++) {
        // Enter critical section
        pthread_mutex_lock(&counter_mutex);

        shared_counter++;
    }
}

```

```
        // Exit critical section
        pthread_mutex_unlock(&counter_mutex);
    }
    printf(" [Thread %ld] Completed %d synchronized increments.\n", thread_id, INCREMEN
TS_PER_THREAD);
    pthread_exit(NULL);
}

int main() {
    pthread_t threads[NUM_THREADS];
    long long expected_counter = (long long)NUM_THREADS * INCREMENTS_PER_THREAD;

    printf("=== Multithreaded Counter: Mutex Synchronization ===\n");
    printf("Number of threads:      %d\n", NUM_THREADS);
    printf("Increments per thread:    %d\n", INCREMENTS_PER_THREAD);
    printf("Expected Final Counter:  %lld\n\n", expected_counter);

    // 1. Initialize mutex
    if (pthread_mutex_init(&counter_mutex, NULL) != 0) {
        perror("pthread_mutex_init failed");
        exit(EXIT_FAILURE);
    }

    printf("Starting worker threads...\n");

    // 2. Spawn worker threads
    for (long t = 0; t < NUM_THREADS; t++) {
        int rc = pthread_create(&threads[t], NULL, worker_thread, (void *)t);
        if (rc != 0) {
            perror("pthread_create failed");
            exit(EXIT_FAILURE);
        }
    }

    // 3. Wait for all threads to finish
    for (int t = 0; t < NUM_THREADS; t++) {
        pthread_join(threads[t], NULL);
    }

    // 4. Destroy mutex
    pthread_mutex_destroy(&counter_mutex);

    long long lost_updates = expected_counter - shared_counter;

    printf("\n=== Execution Results ===\n");
    printf("Expected Counter:  %lld\n", expected_counter);
    printf("Actual Counter:    %lld\n", shared_counter);
    printf("Lost Updates:      %lld\n", lost_updates);

    if (shared_counter == expected_counter) {
        printf("RESULT: MUTEX SYNCHRONIZATION SUCCESS! Counter is 100%% consistent.\n");
    } else {
        printf("RESULT: Inconsistency detected!\n");
    }

    return 0;
}
```

Output:

```
zsh - mutex_counter

rishivarma@MacBook-Air ~ % ./mutex_counter
=== Multithreaded Counter: Mutex Synchronization ===
Number of threads:      4
Increments per thread:  250000
Expected Final Counter: 1000000
Starting worker threads...
[Thread 1] Completed 250000 synchronized increments.
[Thread 3] Completed 250000 synchronized increments.
[Thread 2] Completed 250000 synchronized increments.
[Thread 0] Completed 250000 synchronized increments.
=== Execution Results ===
Expected Counter: 1000000
Actual Counter:   1000000
Lost Updates:     0
RESULT: MUTEX SYNCHRONIZATION SUCCESS! Counter is 100% consistent.
rishivarma@MacBook-Air ~ %
```